

Membres du groupe Mammamia

SIDENNAS Lina

HALLAL Inas

BELKHIR Kamel

////////////////////////////////////

Classe testée : CataloguePizzas

Méthode de test	Actions	Attendu
testAjoutPizzaCatalogue	Nettoyer le catalogue. Créer une pizza valide (Margherita, type Vegetarienne). Ajouter la pizza au catalogue via CataloguePizzas.ajouterPizza(p)	Le catalogue contient exactement une pizza. La pizza ajoutée est présente dans la liste.
testCatalogueStatiquePartage	Nettoyer le catalogue. Créer deux pizzas valides. Ajouter successivement les deux pizzas au catalogue.	Le catalogue contient deux pizzas. L'ordre d'insertion est respecté : la première pizza est à l'indice 0 et la seconde à l'indice 1.

Méthode de test	Actions	Attendu
testCatalogueStatiquePartage	Ajouter plusieurs pizzas consécutivement dans le catalogue statique.	Les pizzas ajoutées précédemment sont conservées. Le catalogue est partagé et commun à toutes les utilisations.

Méthode de test	Actions	Attendu
testAjoutPizzaNull	Nettoyer le catalogue. Ajouter une valeur null au catalogue via CataloguePizzas.ajouterPizza(null).	Le catalogue contient un élément. L'élément ajouté est null, ce qui reflète le comportement actuel de la méthode.

Classe testée : Client

Méthode de test	Actions	Attendu
testConstructeurValide	Créer un client avec un email valide, un mot de passe valide et des informations personnelles valides.	Le client est créé correctement. Les getters retournent les valeurs fournies et la liste des commandes est vide.
testConstructeurEmailNull	Tenter de créer un client avec un email à null.	Une IllegalArgumentException est levée.
testConstructeurEmailVide	Tenter de créer un client avec un email vide ou composé uniquement d'espaces.	Une IllegalArgumentException est levée.
testConstructeurMotDePasseNull	Tenter de créer un client avec un mot de passe null.	Une IllegalArgumentException est levée.
testConstructeurMotDePasseVide	Tenter de créer un client avec un mot de passe vide.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
tDePasseVide	de passe vide.	
testConstructeurInf osNull	Tenter de créer un client avec des informations personnelles null.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testSetMotDePasseVali de	Modifier le mot de passe du client avec une valeur valide.	Le getter du mot de passe retourne la nouvelle valeur.
testSetMotDePasseInva lide	Tenter de modifier le mot de passe avec une valeur invalide (vide).	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testAjoutCommandeVali de	Ajouter une commande valide au client.	La commande est ajoutée et la liste des commandes contient un élément.
testAjoutCommandeNull	Tenter d'ajouter une commande null.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testCommandesPasseesEtE nCours	Ajouter une commande validée et traitée ainsi qu'une commande non traitée au client.	La commande traitée apparaît dans les commandes passées et la commande non traitée apparaît dans les commandes en cours.

Méthode de test	Actions	Attendu
testListeCommandesNonModifiable	Tenter de modifier directement la liste des commandes retournée par le getter.	Une UnsupportedOperationException est levée.

Méthode de test	Actions	Attendu
testEqualsClient	Créer deux clients avec le même email mais des mots de passe différents.	Les deux clients sont égaux et ont le même hashCode.
testNotEqualsClient	Créer deux clients avec des emails différents.	Les deux clients ne sont pas égaux.

Classe testée : Commande

Méthode de test	Actions	Attendu
testCycleCommande	Créer une commande avec un client valide. Ajouter une pizza avec un prix et une quantité. Valider puis traiter la commande.	Le prix total est correctement calculé. L'état de la commande passe successivement à VALIDEE puis à TRAITEE.
testAjoutPizzaApresValidation	Valider une commande puis tenter d'ajouter une pizza.	Une CommandeException est levée.

Méthode de test	Actions	Attendu
testConstructeurClientNull	Tenter de créer une commande avec un client null.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testAjoutPizzaNull	Tenter d'ajouter une pizza null avec une quantité valide.	Une CommandeException est levée.
testAjoutQuantiteInvalide	Tenter d'ajouter une pizza avec une quantité nulle ou négative.	Une CommandeException est levée.
testAjoutMemePizzaDeuxFois	Ajouter deux fois la même pizza avec des quantités différentes.	Les quantités sont cumulées et le prix total est correctement recalculé.

Méthode de test	Actions	Attendu
testAnnulerCommande	Ajouter une pizza à une commande en cours puis annuler la commande.	La commande passe à l'état TRAITEE et la liste des pizzas est vidée.
testAnnulerCommandeApresValidation	Valider une commande puis tenter de l'annuler.	Une CommandeException est levée.

Méthode de test	Actions	Attendu
testMapPizzasNonModifiable	Tenter de modifier la map retournée par getPizzas().	Une UnsupportedOperationException est levée.

Méthode de test	Actions	Attendu
testEqualsCommande	Comparer deux références identiques d'une même commande.	Les deux références sont égales et ont le même hashCode.
testNotEqualsNull	Comparer une commande avec null.	Les deux objets ne sont pas égaux.
testNotEqualsAutreObjet	Comparer une commande avec un objet d'un autre type.	Les deux objets ne sont pas égaux.

Classe testée : Evaluation

Méthode de test	Actions	Attendu
testEvaluationValideSansAuteur	Créer une évaluation avec une pizza valide, une note valide et un commentaire, sans auteur explicite.	L'évaluation est créée correctement. La note, le commentaire et la pizza sont correctement stockés et l'auteur est null.
testEvaluationValideAvecAuteur	Créer une évaluation avec une pizza valide, un auteur valide, une note valide et un commentaire.	Tous les attributs sont correctement initialisés et accessibles via les getters.
testEvaluationCommentaireNull	Créer une évaluation avec un commentaire null.	L'évaluation est créée. Le commentaire est null et aucune exception n'est levée.
testNoteMinimale	Créer une évaluation avec une note minimale égale à 0.	L'évaluation est créée et la note est correctement enregistrée.
testNoteMaximale	Créer une évaluation avec une note maximale égale à 5.	L'évaluation est créée et la note est correctement enregistrée.

Méthode de test	Actions	Attendu
testPizzaNulle	Tenter de créer une évaluation avec une pizza null.	Une IllegalArgumentException est levée.
testNoteNegative	Tenter de créer une évaluation avec une note négative.	Une IllegalArgumentException est levée.
testNoteSuperieure	Tenter de créer une évaluation avec une	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
eureA5	note strictement supérieure à 5.	
testAuteurNull	Tenter de créer une évaluation avec un auteur null.	Une IllegalArgumentException est levée.
testAuteurVide	Tenter de créer une évaluation avec un auteur vide.	Une IllegalArgumentException est levée.
testAuteurEspaces	Tenter de créer une évaluation avec un auteur composé uniquement d'espaces.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testToString	Appeler la méthode toString() sur une évaluation valide.	La chaîne retournée contient le nom de la pizza, l'auteur, la note et le commentaire.

Classe testée : GestionCommande

Méthode de test	Actions	Attendu
testSingleton	Appeler plusieurs fois la méthode getInstance().	La même instance de GestionCommande est retournée à chaque appel.

Méthode de test	Actions	Attendu
testAjoutCommandeValidee	Créer une commande, la valider puis l'ajouter via ajouterCommandeValidee.	La commande apparaît dans la liste des commandes à

Méthode de test	Actions	Attendu
		traiter.
testRefusCommandeNonValidee	Créer une commande non validée et tenter de l'ajouter.	La commande n'est pas ajoutée et la liste des commandes à traiter reste vide.
testAjoutCommandeNulle	Tenter d'ajouter une commande null.	Aucune exception n'est levée et aucune commande n'est ajoutée.

Méthode de test	Actions	Attendu
testTraitementAutomatique	Ajouter une commande validée puis récupérer les commandes non traitées.	Les commandes récupérées sont automatiquement marquées comme TRAITEE.
testNettoyageCommandesATraiter	Ajouter une commande validée puis appeler la récupération des commandes non traitées.	La liste des commandes à traiter est vidée après le traitement.

Méthode de test	Actions	Attendu
testGetCommandesClient	Ajouter et traiter des commandes appartenant à différents clients.	Seules les commandes du client demandé sont retournées.
testClientSansCommandes	Récupérer les commandes d'un client n'ayant jamais passé de commande.	Une liste non nulle et vide est retournée.

Classe testée : Ingredient

Méthode de test	Actions	Attendu
testCreationValide	Créer un ingrédient avec un nom valide et un prix positif.	L'ingrédient est créé correctement. Le nom et le prix sont accessibles via les getters.
testNomNullInterdit	Tenter de créer un ingrédient avec un nom null.	Une IllegalArgumentException est levée.
testNomVideInterdit	Tenter de créer un ingrédient avec un nom vide ou composé uniquement d'espaces.	Une IllegalArgumentException est levée.
testPrixNegatifInterdit	Tenter de créer un ingrédient avec un prix négatif.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testSetPrix	Modifier le prix d'un ingrédient avec une valeur valide.	Le nouveau prix est correctement enregistré.
testSetPrixNegatif	Tenter de modifier le prix avec une valeur négative.	Une IllegalArgumentException est levée.

Classe testée : Pizzaiolo

Méthode de test	Actions	Attendu
testCreerIngrédient	Créer un ingrédient valide puis tenter des créations invalides (doublon, nom nul, nom vide, prix nul).	La création valide réussit. Les cas invalides retournent les codes d'erreur attendus.
testChangerPrixIngrédient	Créer un ingrédient puis modifier son prix avec différentes valeurs.	Le changement de prix valide réussit. Les valeurs invalides retournent les codes d'erreur correspondants.

Méthode de test	Actions	Attendu
testCreerPizza	Créer une pizza valide puis tenter des créations invalides (doublon, nom nul, nom vide).	La pizza valide est créée. Les créations invalides retournent null.
testAjouterEtRetire rIngredientPizza	Ajouter puis retirer des ingrédients d'une pizza existante.	Les ajouts et retraits valides réussissent. Les cas invalides retournent les codes d'erreur appropriés.

Méthode de test	Actions	Attendu
testInterdictionI nгредиent	Interdire un ingrédient pour un type de pizza puis tenter de l'ajouter.	L'ingrédient est interdit et ne peut pas être ajouté à la pizza.
testVerifierIngred ientsPizza	Ajouter un ingrédient à une pizza puis l'interdire et vérifier les interdictions.	La liste des ingrédients interdits contient l'ingrédient concerné.

Méthode de test	Actions	Attendu
testPrixPizz a	Calculer le prix minimal d'une pizza, fixer un prix valide puis un prix invalide.	Le prix valide est accepté et correctement enregistré. Le prix invalide est refusé.

Méthode de test	Actions	Attendu
testCommandesNo nTraitees	Ajouter une commande validée puis récupérer les commandes non traitées via le pizzaïolo.	La commande est automatiquement traitée et retournée dans la liste.

Méthode de test	Actions	Attendu
testStatistiquesSimple	Créer une pizza, passer une commande, la traiter puis calculer les statistiques associées.	Les commandes traitées sont récupérées et le bénéfice de la commande est correctement calculé.

Classe testée : Pizza

Méthode de test	Actions	Attendu
testConstructeurValide	Créer une pizza avec un nom valide et un type valide.	La pizza est créée correctement. Le nom, le type, le prix de vente et la note moyenne sont initialisés correctement.
testConstructeurInvalid	Tenter de créer une pizza avec un nom vide ou un type null.	Une IllegalArgumentException est levée.

Méthode de test	Actions	Attendu
testAjouterIngredient	Ajouter un ingrédient valide à une pizza.	L'ingrédient est ajouté et apparaît dans la liste des ingrédients.
testAjouterIngredientNull	Tenter d'ajouter un ingrédient null.	Une IllegalArgumentException est levée.
testRetirerIngredient	Ajouter puis retirer un ingrédient d'une pizza.	Le retrait valide retourne true. Un second retrait ou un retrait avec null retourne false.

Méthode de test	Actions	Attendu
testPrixMinimal	Ajouter plusieurs ingrédients à une pizza et calculer le prix minimal.	Le prix minimal correspond à la somme des prix des ingrédients multipliée par le coefficient défini.
testSetPrixVenteValide	Définir un prix de vente supérieur ou égal au prix minimal.	Le prix de vente est correctement enregistré.
testSetPrixVenteInvalide	Tenter de définir un prix de vente inférieur au prix minimal.	Une IllegalArgumentException est levée.
testBeneficeUnitaire	Définir un prix de vente supérieur au prix minimal puis calculer le bénéfice.	

Méthode de test	Actions	Attendu
testPrixMinimal	Ajouter plusieurs ingrédients à une pizza et calculer le prix minimal.	Le prix minimal correspond à la somme des prix des ingrédients multipliée par le coefficient défini.
testSetPrixVenteValide	Définir un prix de vente supérieur ou égal au prix minimal.	Le prix de vente est correctement enregistré.
testSetPrixVenteInvalide	Tenter de définir un prix de vente inférieur au prix minimal.	Une IllegalArgumentException est levée.
testBeneficeUnitaire	Définir un prix de vente supérieur au prix minimal puis calculer le bénéfice.	Le bénéfice unitaire correspond à la différence entre le prix de vente et le prix minimal.

Méthode de test	Actions	Attendu
testEvaluationValide	Ajouter une évaluation valide à une pizza.	L'évaluation est ajoutée et la note moyenne est correctement mise à jour.
testNoteMoyenneSansEvaluation	Calculer la note moyenne d'une pizza sans évaluation.	La note moyenne est égale à 0.

Méthode de test	Actions	Attendu
testEvaluationInv alide	Tenter d'ajouter une évaluation null ou une évaluation associée à une autre pizza.	Une IllegalArgumentException est levée.

Classe testée : ServiceClient

Méthode de test	Actions	Attendu
testInscription EtConnexion	Inscrire un client puis tenter une seconde inscription avec le même email. Tester ensuite la connexion avec des identifiants valides et invalides.	L'inscription valide réussit. L'email déjà utilisé est refusé. La connexion réussit avec les bons identifiants et échoue sinon.
testDeconnexion	Inscrire et connecter un client, puis appeler la méthode de déconnexion.	Après déconnexion, toute tentative de démarrer une commande déclenche une NonConnecteException.

Méthode de test	Actions	Attendu
testDebuterComm ande	Inscrire et connecter un client puis démarrer une commande.	Une commande est créée avec l'état CREEE.
testDebuterComm andeSansConnexion	Tenter de démarrer une commande sans être connecté.	Une NonConnecteException est levée.
testAjouterEtValid erCommande	Créer une commande, y ajouter une pizza, la valider puis la faire traiter par le pizaiolo.	La commande apparaît dans les commandes passées du client et son état est TRAITEE.
testAnnulerComm ande	Démarrer une commande puis l'annuler.	La commande n'est pas validée et son état est modifié conformément aux règles métier.

Méthode de test	Actions	Attendu
testAjouterEtValiderCommande	Valider puis traiter une commande.	La commande est accessible via getCommandePassees().

Méthode de test	Actions	Attendu
testFiltresCatalogue	Appliquer un filtre par type de pizza puis un filtre par prix sur le catalogue.	Seules les pizzas correspondant aux filtres appliqués sont retournées.

Méthode de test	Actions	Attendu
testAjouterEvaluationValide	Commander une pizza, valider la commande puis ajouter une évaluation.	L'évaluation est ajoutée avec succès et la note moyenne de la pizza est mise à jour.
testAjouterEvaluationSansCommande	Tenter d'ajouter une évaluation pour une pizza non commandée par le client.	Une CommandeException est levée.

Classe testée : InformationPersonnelle

Méthode de test	Actions	Attendu
testConstructeurBasique	Créer une instance avec le constructeur basique (nom, prénom).	Le nom et le prénom sont correctement initialisés. L'âge est initialisé à 0 et l'adresse n'est pas nulle.
testConstructeurCompleto	Créer une instance avec le constructeur complet (nom, prénom, adresse, âge).	L'âge et l'adresse correspondent aux valeurs fournies.

Méthode de test	Actions	Attendu
testSetAge Valide	Modifier l'âge avec une valeur positive valide.	L'âge est mis à jour correctement.
testSetAge Invalide	Modifier l'âge avec une valeur négative.	L'âge est ramené à 0 conformément au comportement défini.

Méthode de test	Actions	Attendu
testSetAdresseNull	Tenter de définir une adresse null.	L'adresse n'est pas null après l'appel du setter.